CRAFT: Exploiting Precharge Cost Asymmetry for Adaptive DRAM Row Buffer Management

First Author[1[0000−1111−2222−3333]], Second Author[2,3[1111−2222−3333−4444]], and Third Author[3[2222−−3333−4444−5555]]

[1] Princeton University, Princeton NJ 08544, USA
[2] Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com
http://www.springer.com/gp/computer-science/lncs
[3] ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Abstract. DRAM row buffer management presents a fundamental trade-off between exploiting row buffer locality and minimizing conflict penalties. The optimal policy varies across workloads, execution phases, and individual address ranges. Existing adaptive schemes, however, face a persistent tension between hardware complexity and management effectiveness.

This paper presents CRAFT, a lightweight, cost-aware adaptive timeout scheme for DRAM row buffer management. CRAFT exploits the observation that the two failure modes of speculative precharge incur qualitatively different performance costs and translates this cost asymmetry into differentiated per-bank timeout adjustments. Incorrect speculative precharges increase the timeout through exponential backoff, whereas conflicts reduce it with a smaller step proportional to the avoidable overhead. Beyond this core feedback loop, three precharge-outcome-based refinements further improve adaptation precision without additional hardware structures. The entire mechanism requires no prediction tables, feature extraction, or learned models.

We evaluate CRAFT using a cycle-accurate simulator across 12 memory-intensive workloads from graph analytics and scientific computing. CRAFT achieves IPC improvements of 7.73%, 3.10%, and 2.84% over ABP, DYMPL, and INTAP, respectively. The total storage overhead is 140 bytes per channel, representing a 24× reduction relative to DYMPL and over 140× reduction relative to ABP. These results indicate that the cost asymmetry inherent in precharge outcomes is an effective basis for lightweight row buffer management in our evaluated setting.

Keywords: DRAM row buffer management · page policy · memory controller · feedback-driven adaptation

1 Introduction

Main memory latency remains a primary performance bottleneck in modern processor systems [1–3]. A critical factor governing this latency is how the mem-

ory controller manages the *row buffer* in each DRAM bank, the internal sense-amplifier array that caches the most recently activated row. Holding a row open (the *open-page* policy) maximizes the chance of serving subsequent accesses to the same row at low latency, but risks expensive conflicts when a subsequent request targets a different row. Closing immediately after each access (the *close-page* policy) eliminates such conflicts but forfeits all potential row locality. This fundamental tradeoff makes row buffer management one of the most extensively studied yet persistently challenging aspects of memory controller design [4, 5].

The optimal page policy varies across workloads, across execution phases, and even across targeted address ranges. Neither open-page nor close-page dominates across all workloads. Open-page loses up to 11% IPC on workloads with frequent row buffer conflicts. Close-page sacrifices up to 28% IPC on workloads with high row buffer locality (Fig. 1(a)). Within a single program execution, row buffer hit rate shifts dramatically between program phases. The row buffer hit rate of `mcf` transitions from above 90% to below 10%, while `PageRank` follows the inverse trajectory (Fig. 1(b)). Beyond temporal variation, row buffer locality also differs substantially across banks within a single channel. Under the open-page policy, `omnetpp` exhibits a per-bank row buffer hit rate coefficient of variation of 1.51, indicating pronounced inter-bank disparity. These observations suggest that effective row buffer management should adapt per bank and per execution phase.

Achieving per-bank, phase-adaptive row buffer management with low hardware overhead remains an open problem. Prior adaptive schemes have addressed aspects of this challenge but face a fundamental tradeoff between hardware complexity and adaptation precision. ABP [6] requires approximately 20 KB per channel for per-row prediction tables. DYMPL [7] employs a perceptron-based classifier with 3.39 KB per channel and requires machine-learning inference on the critical path. INTAP [8] adjusts a per-bank timeout using approximately 200 bytes per channel. It applies the same fixed step size to both incorrect speculative precharges and conflicts. This symmetric adjustment does not distinguish between the penalty induced by speculative precharge decisions and the inherent cost of the workload's access pattern.

This paper presents CRAFT, a lightweight, cost-aware adaptive timeout scheme for DRAM row buffer management. Among the four observable outcomes of timeout-based speculative precharge, CRAFT focuses on three feedback-relevant cases: correct speculative precharge, incorrect speculative precharge, and conflict. It translates their asymmetric performance costs into differentiated per-bank timeout adjustments. Incorrect speculative precharges increase the timeout through exponential backoff, whereas conflicts reduce it with a smaller proportional step. The resulting mechanism requires no prediction tables, feature extraction, or learned models. The complete cost analysis and design rationale are presented in Section 3.1.

The contributions of this paper are as follows:

1. We identify that incorrect speculative precharges and conflicts incur qualitatively different performance costs, and propose CRAFT, a feedback-driven

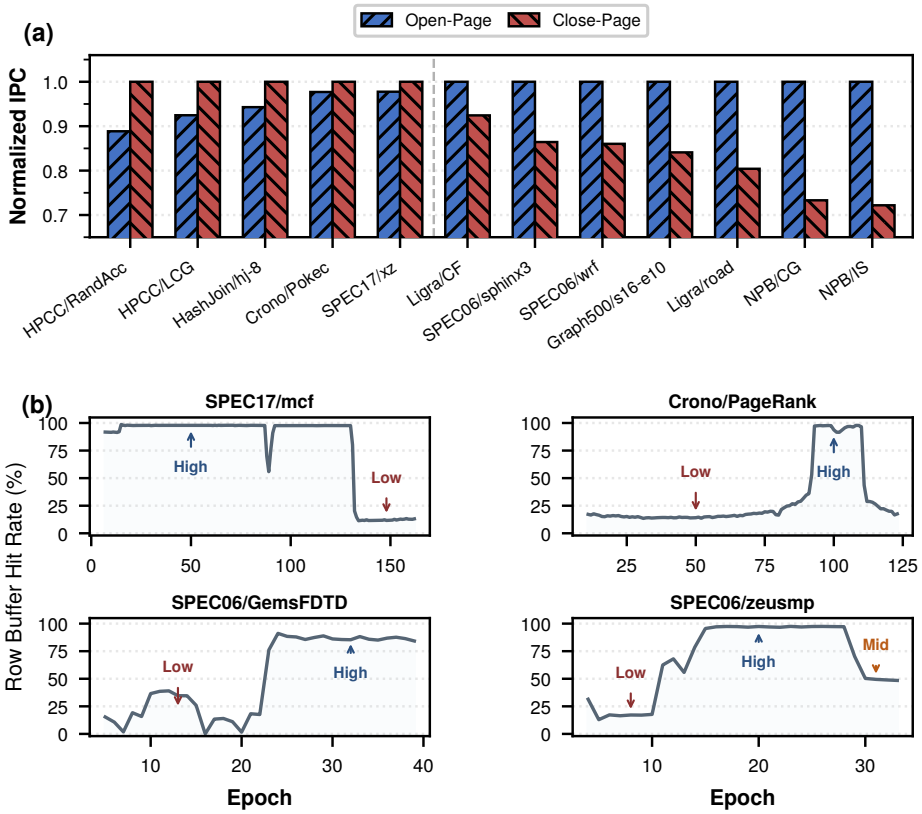

Fig. 1. (a) Normalized IPC of open-page and close-page policies across 12 memory-intensive workloads, normalized to the better static policy per workload. Neither static policy dominates across all workloads. (b) Row buffer hit rate across execution epochs under the open-page policy for four representative workloads. Each workload exhibits distinct phase transitions between high- and low-locality regimes.

scheme that translates this cost asymmetry into differentiated per-bank timeout adjustments requiring only 140 bytes per channel, representing over 140× storage reduction compared to ABP.

2. Through ablation analysis, we show that precharge outcomes provide an effective basis for timeout adaptation in our evaluated setting, without requiring auxiliary runtime metrics.

3. We evaluate CRAFT on 12 memory-intensive workloads under a DDR5-4800 configuration. CRAFT improves IPC relative to ABP, DYMPL, and INTAP across the evaluated workloads.

2 Background

2.1 DRAM Row Buffer Fundamentals

Modern DRAM organizes each bank around a *row buffer*, an array of sense amplifiers that latches the most recently activated row. As illustrated in Fig. 2(a), access latency depends on the row buffer state: a hit incurs only t_{CAS}, a miss requires $t_{\mathrm{RCD}} + t_{\mathrm{CAS}}$, and a conflict requires precharge followed by activation, totaling $t_{\mathrm{RP}} + t_{\mathrm{RCD}} + t_{\mathrm{CAS}}$. An *open-page* policy keeps the row buffer open to exploit hits; a *close-page* policy precharges immediately to avoid conflicts. Neither static policy dominates universally, as shown in Fig. 1.

Timeout-based speculative precharge. Timeout-based policies occupy a middle ground between open-page and close-page behavior. The memory controller starts a countdown timer when a row becomes idle. If the timer expires before the next request, the controller speculatively precharges the bank. A short timeout approximates close-page behavior. A long timeout approximates open-page behavior. This speculation produces one of four outcomes, illustrated in Fig. 2(b):

1. **Correct speculative precharge.** The next request targets a different row; the speculative precharge correctly anticipated the end of locality. The effective cost is zero.
2. **Incorrect speculative precharge.** The next request targets the same row. The controller must reactivate the row. Both the precharge and the reactivation are entirely unnecessary operations that would not occur under the open-page baseline.
3. **Conflict.** The next request targets a different row, but the timeout has not yet expired. The controller issues an on-demand precharge and then activates the target row. The activation cost is inherent to the access pattern. Only the opportunity to overlap precharge with idle bank cycles is lost.
4. **Row buffer hit.** The next request targets the same row and arrives before the timeout, incurring zero additional cost. Because no precharge is issued in this case, CRAFT's feedback loop operates on the first three outcomes only.

The key challenge is to set the timeout value adaptively, per bank and over time, to match the evolving access pattern.

2.2 Limitations of Existing Adaptive Schemes

Several adaptive row buffer management schemes have been proposed. Table 1 summarizes three representative baselines.

ABP [6] requires approximately 20 KB per channel for a set-associative per-row prediction table, which is impractical for DDR5 controllers with 32 banks per channel. DYMPL [7] employs a perceptron over seven features with a 512-entry page row table (3.39 KB per channel); it does not directly observe the

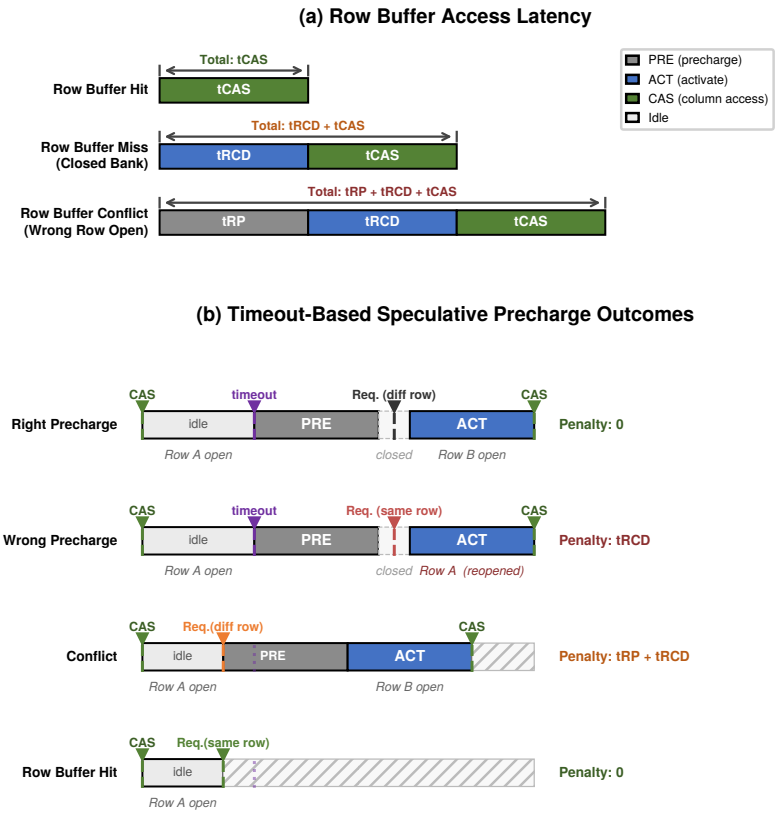

Fig. 2. Row buffer access scenarios and timeout precharge outcomes. (a) Three row buffer states at the time of a memory request. A row buffer hit incurs only t_{CAS}. A closed-bank access requires $t_{\mathrm{RCD}} + t_{\mathrm{CAS}}$. A row buffer conflict pays $t_{\mathrm{RP}} + t_{\mathrm{RCD}} + t_{\mathrm{CAS}}$. (b) Four outcomes of timeout-triggered speculative precharge. CRAFT's feedback loop uses the three outcomes involving a precharge event.

Table 1. Comparison of adaptive row buffer management baselines.

Scheme	Decision Mechanism	Storage/Ch	Key Limitation
ABP	Per-row access count table	∼20 KB	High storage; binary prediction
DYMPL	Perceptron + page row table	3.39 KB	Indirect features; no cost-aware adaptation
INTAP	Mistake counter, fixed step	∼200 B	Symmetric step; no cost differentiation

cost consequences of precharge decisions. INTAP [8] adjusts a per-bank timeout through a mistake counter, but applies symmetric fixed-step adjustments that treat incorrect speculative precharges and conflicts identically.

These scheme-specific limitations differ in nature, but all three baselines share a deeper deficiency. None of them distinguishes precharge outcomes by the nature of the waste they produce. This distinction directly motivates CRAFT's cost-aware timeout adaptation, analyzed in detail in Section 3.1.

3 CRAFT Design

CRAFT exploits the cost asymmetry of precharge outcomes to adaptively manage DRAM row buffers. This section describes the core feedback loop (Section 3.1), three precharge-outcome-based refinements (Section 3.2), and hardware implementation (Section 3.3).

3.1 Core Feedback Loop

CRAFT maintains a per-bank timeout counter that specifies the number of idle cycles after the last column access before the memory controller speculatively issues a precharge command. CRAFT detects the three precharge outcomes at two distinct points. A conflict occurs when a new request for a different row arrives while the timeout has not yet expired, indicating the timeout is too long. Incorrect and correct speculative precharge outcomes are identified at the bank activation following a timeout-triggered precharge. A same-row activation indicates an incorrect speculative precharge and signals that the timeout was too short. A different-row activation indicates a correct speculative precharge and confirms the current timeout value is appropriate. These three outcomes provide a direct feedback signal for timeout adaptation. CRAFT exploits the different performance costs of these outcomes to derive both the direction and the magnitude of each adjustment.

CRAFT adjusts the per-bank timeout value in response to these outcomes according to three design principles.

(a) Cost-driven asymmetric step sizes. Timeout-based speculative precharge serves both sides of the row buffer management trade-off. Keeping a row open allows same-row accesses to be served as row buffer hits. Issuing a timely precharge overlaps precharge overhead with idle bank cycles, reducing the effective penalty for subsequent different-row accesses. An incorrect speculative precharge eliminates the first benefit entirely. Both the precharge command and the subsequent reactivation are unnecessary operations that would not occur under the open-page baseline. The total controller-attributable waste therefore amounts to $t_{\mathrm{RP}} + t_{\mathrm{RCD}}$.

Conflicts present a fundamentally different cost model. The target row differs from the buffered row, so the precharge and subsequent activation are inherent

to the access pattern regardless of the timeout decision. Only the timing constitutes a controller-attributable error. A correctly timed precharge would have overlapped t_{RP} with idle bank cycles. The avoidable overhead is therefore limited to the missed overlap opportunity of t_{RP}.

The ratio of avoidable overhead between these two outcomes is $(t_{\mathrm{RP}}+t_{\mathrm{RCD}})/t_{\mathrm{RP}}$. Under our DDR5-4800 configuration with $t_{\mathrm{RP}} = t_{\mathrm{RCD}} = 40$ cycles, this ratio evaluates to 2:1. CRAFT mirrors this asymmetry in its adjustment magnitudes. The increment for incorrect speculative precharges uses a base value of `BASE_STEP` = 50 cycles, slightly larger than t_{RP}, so each increase produces an observable change in the timeout. The decrement for conflicts is scaled by the factor $t_{\mathrm{RP}}/(t_{\mathrm{RP}}+t_{\mathrm{RCD}})$ and evaluates to 25 cycles under the same configuration.

This asymmetry creates a deliberate upward bias: under equal frequencies of incorrect speculative precharges and conflicts, net timeout increases accumulate, reflecting the 2:1 cost ratio. T_{MAX} bounds this bias at 3200 cycles; conflicts and timeout reduction after consecutive correct speculative precharges then provide downward pressure.

(b) Exponential backoff. CRAFT applies exponential backoff to consecutive incorrect speculative precharges by left-shifting the base step:

$$step = \texttt{BASE_STEP} \times 2^{\min(reopen_streak,\ \texttt{SHIFT_CAP})} \tag{1}$$

The *reopen_streak* counter tracks consecutive incorrect speculative precharges, is capped at 8, and resets to zero upon a correct speculative precharge or a conflict. This mechanism enables rapid convergence during sustained high-locality phases. The timeout can reach the upper bound within a small number of feedback events.

(c) Continuous adjustment range. The timeout adjusts within $[T_{\mathrm{MIN}}, T_{\mathrm{MAX}}] = [50, 3200]$ cycles. T_{MIN} matches `BASE_STEP` to prevent degeneration to close-page behavior; T_{MAX} prevents indefinitely open rows. The exponential backoff traverses the full range in as few as six escalation events.

Algorithm 1 presents the complete pseudocode of the core feedback loop.

3.2 Precharge-Path Refinements

The core feedback loop provides the primary adaptive capability. The following three refinements exploit additional information available from precharge outcomes to improve adaptation precision. All three are lightweight and introduce no additional hardware structures beyond simple counters.

(a) Timeout Reduction After Consecutive Correct Speculative Precharges (RS). After four or more consecutive correct speculative precharges, the timeout decrements by half the conflict decrement. This gradual reduction prevents timeout values from remaining at elevated levels after a high-locality phase has ended. Without RS, a bank that transitions from a high-locality phase to a low-locality

Algorithm 1 CRAFT Core Feedback Loop

```
On each incoming bank request (while timeout active):
if new_row ≠ open_row then                                   ▷ Conflict
    step ← BASE_STEP × t_RP/(t_RP + t_RCD)
    timeout ← timeout − step                                 ▷ Reduce timeout
    reopen_streak ← 0
    right_streak ← 0
    timeout ← clamp(timeout, T_MIN, T_MAX)
end if

On each bank ACT event (following timeout precharge):
if prev_closed_by_timeout then
    if new_row = prev_row then                               ▷ Incorrect speculative precharge
        step ← BASE_STEP ≪ min(reopen_streak, SHIFT_CAP)
        timeout ← timeout + step                             ▷ Increase timeout
        reopen_streak ← min(reopen_streak + 1, 7)
        right_streak ← 0
    else                                                     ▷ Correct speculative precharge
        reopen_streak ← 0
        right_streak ← right_streak + 1
    end if
    timeout ← clamp(timeout, T_MIN, T_MAX)
end if
```

phase would maintain an unnecessarily large timeout until a conflict explicitly triggers a timeout reduction. RS provides a proactive path for timeout reduction in the absence of explicit conflict signals. RS adds three bits per bank for the *right_streak* counter.

(b) Read/Write Cost Differentiation (RW). An incorrect speculative precharge blocking a read stalls the processor pipeline, while one blocking a write is absorbed by the write buffer. CRAFT exploits this asymmetry by halving the increment for write incorrect speculative precharges and doubling the decrement for read conflicts. RW requires no additional storage.

(c) Streak Decay (SD). Each correct speculative precharge decrements *reopen_streak* by one, preventing a historically elevated streak from causing disproportionately large increments much later. SD requires no additional storage.

Section 5.4 validates the effectiveness of these three refinements through an ablation study.

3.3 Hardware Overhead

Each bank maintains five fields totaling 35 bits: a 12-bit timeout value, two 3-bit streak counters, a 16-bit previous-row address, and a 1-bit timeout-precharge flag. For 32 banks per channel, the total overhead is 140 bytes per channel. CRAFT requires no specialized hardware structures; the critical-path computation is a single comparison and a single addition or subtraction.

4 Methodology

4.1 Simulation Infrastructure

We evaluate CRAFT using ChampSim [9] integrated with DRAMSim3 [10]. We implement CRAFT and all three baselines within the same simulation infrastructure and DRAM controller model. All schemes share the same processor configuration, cache hierarchy, DRAM timing parameters, address mapping, command queue sizes, traces, warmup intervals, and measurement intervals. Table 2 summarizes the common configuration.

Table 2. Simulation configuration.

Component	Parameter	Value
Processor	Frequency	4 GHz
	Pipeline width	6
	ROB entries	350
	Branch predictor	TAGE-SC-L
Caches	L1 (I/D): Size / Assoc. / Latency	32 KB / 8-way / 4 cycles
	L2: Size / Assoc. / Latency	1 MB / 8-way / 10 cycles
	LLC: Size / Assoc. / Latency	4 MB / 16-way / 20 cycles
DRAM	Standard	DDR5-4800
	Channels / Ranks / Banks per channel	4 / 1 / 32
	Rows per bank	65,536
	t_{CL} / t_{RCD} / t_{RP} (DRAM cycles)	40 / 40 / 40
	Address mapping	rorababgchco
	Command queue (per bank)	8 entries

4.2 Workloads

We select 12 memory-intensive workloads from three benchmark suites. Graph workloads are drawn from LIGRA [11] and CRONO [12] using three SNAP graphs [13]: roadNet-CA, higgs, and soc-pokec. Scientific computing workloads are sphinx3 and wrf from SPEC CPU2006 [14]. Representative phases are identified using SimPoint [15] with maxK = 30 and 100M-instruction intervals; each simulation point uses 20M instructions for warmup and 80M instructions for measurement. The suite is intentionally biased toward memory-intensive single-core behavior, so we interpret the results within that target setting.

4.3 Baselines

We compare CRAFT against ABP [6], DYMPL [7], and INTAP [8]. All baselines are implemented in the same infrastructure under the DDR5-4800 configuration in Table 2. The controller scheduling policy, address mapping, queue capacities, and trace inputs remain fixed across schemes; only the row buffer management logic differs.

4.4 Metrics

We report three metrics. **IPC** is the primary performance metric; we report per-workload improvement relative to each baseline. **Read row buffer hit rate** measures the fraction of read requests served from the currently activated row; write hit rates are excluded as writes complete in the write buffer of the memory controller. **Average read latency** is the mean DRAM service time for read requests in DRAM cycles.

5 Evaluation

This section evaluates CRAFT across four dimensions: system-level IPC performance (Section 5.1), DRAM-level performance analysis (Section 5.2), timeout adaptation behavior (Section 5.3), and an ablation study of individual design components (Section 5.4).

5.1 IPC Performance

Figure 3 presents the per-workload IPC improvement of CRAFT over each baseline. CRAFT achieves geometric mean IPC improvements of 7.73% over ABP, 3.10% over DYMPL, and 2.84% over INTAP. CRAFT improves IPC on all 12 workloads relative to all three baselines, with gains ranging from 1.61% to 12.20%.

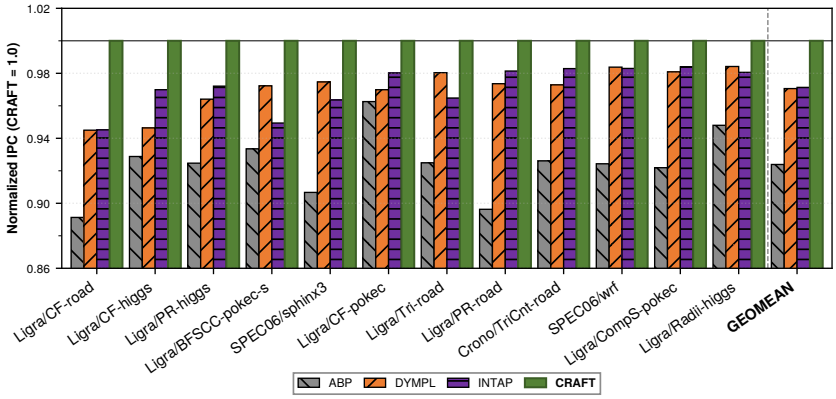

Fig. 3. Normalized IPC across 12 workloads (CRAFT = 1.0). CRAFT improves IPC relative to all three baselines on the evaluated workloads. The geometric mean improvements are 7.73%, 3.10%, and 2.84% over ABP, DYMPL, and INTAP, respectively.

The largest gains over ABP occur in graph traversal workloads such as CF and PageRank (7.1% to 12.2%), where locality changes abruptly across phases. Graph analysis workloads and the two scientific workloads also benefit, indicating that the per-bank feedback loop adapts across several locality regimes. Section 5.3 analyzes the corresponding timeout distributions.

5.2 DRAM-Level Performance Analysis

CRAFT achieves the highest read row buffer hit rate on all 12 workloads, surpassing the best-performing baseline by an average of 5.62% (largest gains: CF/roadNet-CA +9.25%, PageRank/roadNet-CA +9.12%, sphinx3 +7.76%). CRAFT also achieves the lowest average read latency, with a mean reduction of 2.74%. Write row buffer hit rates remain nearly identical across all policies, indicating that the gain primarily comes from the read path.

5.3 Timeout Distribution

Figure 4 presents the timeout value distribution for each workload. We partition the timeout range into Low [50, 800), Mid [800, 2000), and High [2000, 3200], corresponding to close-page-like, intermediate, and open-page-like behavior. Three adaptation patterns emerge.

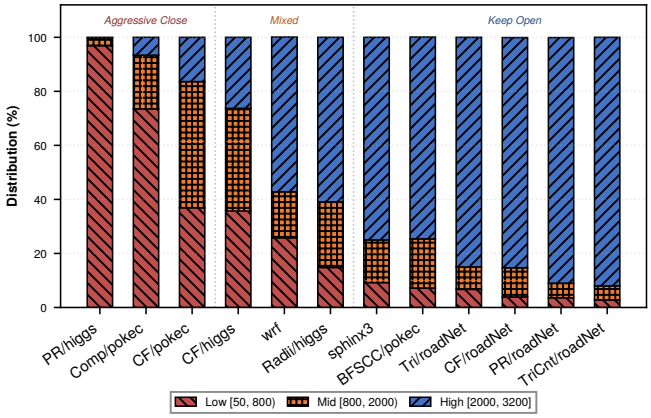

Fig. 4. Timeout value distribution across 12 workloads, sorted from close-page-like (left) to open-page-like (right). CRAFT adapts to three distinct behavioral regimes without any explicit mode selection.

Close-Page-Like. PageRank/higgs concentrates 96.9% of observations in the Low range, and Components-Shortcut/pokec follows a similar pattern.

Gradual Transition. CF/higgs distributes values across all three ranges, reflecting a shift from conflict-dominated early phases to locality-rich later phases.

Open-Page-Like. The roadNet-CA workloads concentrate overwhelmingly in the High range. PageRank on roadNet-CA yields 90.9% High versus 96.9% Low on higgs, confirming that adaptation reflects the runtime access pattern rather than algorithm identity.

5.4 Ablation Study

We conduct an ablation study to quantify the contribution of each design component. Table 3 defines each variant. Figure 5 compares eight CRAFT variants.

Table 3. Ablation study variants.

Variant	Components	Description
BASE	Core loop only	Cost-asymmetric step sizes + exponential backoff
RS	BASE + Right Streak	Timeout reduction after consecutive correct speculative precharges
RW	BASE + Read/Write	Differentiated step sizes by command type
SD	BASE + Streak Decay	Gradual decay of *reopen_streak* on correct speculative precharges
PRECHARGE	BASE + RS + RW + SD	All three precharge-outcome-based refinements combined
PR	BASE + Phase Reset	Reset timeout to initial value on conflict
QDSD	BASE + Queue-Depth	Scale the conflict decrement by queue occupancy
ALL	PRECHARGE + PR + QDSD	All precharge-outcome-based and conflict-based heuristics combined

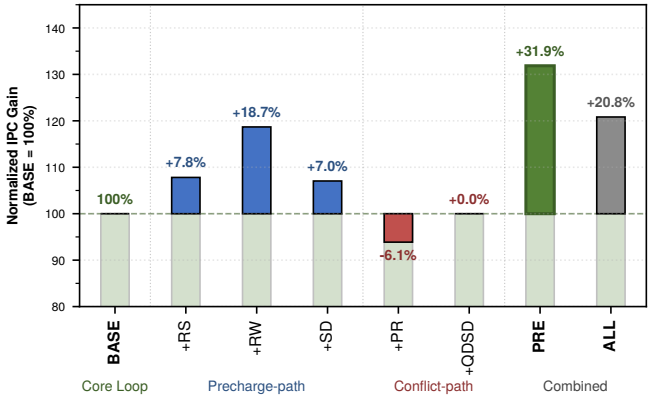

Fig. 5. Ablation study of CRAFT design components. Green bars correspond to the core feedback loop and the PRECHARGE configuration. Blue bars show individual precharge-outcome-based enhancements. Red bars show individual conflict-based heuristics. The dashed line marks the BASE level.

Each variant's geometric mean IPC gain over the best baseline is normalized to the core feedback loop's gain.

BASE (cost-asymmetric step sizes + exponential backoff) accounts for 76% of the final improvement, indicating that outcome-aware feedback provides a strong signal in our evaluated setting. The PRECHARGE variant (BASE + RS + RW + SD) achieves 131.9% of BASE's gain; RW is the strongest individual component at 118.7%, while RS and SD reach 107.8% and 107.0%, respectively. Conflict-based heuristics (PR and QDSD) do not improve performance: individually they reach 93.9% and 100.0% of BASE, and the ALL variant reaches 120.8%, an 11.1% drop from PRECHARGE. In this setting, the three-way precharge outcome classification appears to capture most of the useful feedback information.

6 Related Work

Static page policies set a fixed open or closed behavior for all accesses. JEDEC [16] defines per-command auto-precharge, and Minimalist Open-page [17] closes a row after a fixed hit count. Predictor-based schemes improve on these by inferring per-row reuse from access history or address signals [6, 18]; their storage cost

scales with the number of tracked rows. Adaptive schemes reduce this overhead by operating at the bank granularity. INTAP [8] adjusts a per-bank timeout through a fixed-step mistake counter but applies identical adjustments to both failure modes of speculative precharge. DYMPL [7] applies a perceptron over indirect features and [19] uses reinforcement learning; both require auxiliary model state and do not directly observe precharge outcome cost. CRAFT distinguishes the two failure modes by their cost and uses this asymmetry to set adjustment magnitudes, without any additional model state.

A substantial body of work addresses DRAM performance from orthogonal angles. Scheduling algorithms such as FR-FCFS [20], stall-time fair scheduling [21], Thread Cluster Memory Scheduling [22], and Staged Memory Scheduling [23] govern request issue order within the command queue; the virtual write queue [24] coordinates write-back policy with row management. Multi-core interference proposals [25, 26] partition bandwidth or estimate slowdown to improve fairness across applications. At the device level, SALP [27] and Tiered-Latency DRAM [28] reduce access overhead through bank and subarray organization, RAIDR [29] reduces refresh frequency through retention-aware scheduling, and Udipi et al. [30] propose energy-aware rank arrangements. These directions are complementary to CRAFT, which operates within the memory controller policy layer and requires no changes to the DRAM device or scheduler.

7 Conclusion

This paper presents CRAFT, a lightweight row buffer management scheme that exploits the cost asymmetry between the two failure modes of speculative precharge. CRAFT combines asymmetric timeout updates with exponential backoff and requires only 140 B of storage per channel.

Across 12 memory-intensive workloads, CRAFT improves IPC by 7.73%, 3.10%, and 2.84% over ABP, DYMPL, and INTAP, respectively. CRAFT also improves average read row buffer hit rate by 5.62% over the best baseline and reduces average read latency by 2.74%. Our ablation study shows that precharge-outcome-based refinements improve adaptation precision, while additional conflict-based heuristics do not help in this setting.

These results support CRAFT as a practical option for the evaluated single-core DDR5 configuration. The current evaluation emphasizes memory-intensive single-core workloads, and broader multi-core and cross-standard validation remains future work.

Acknowledgments. [Acknowledgments to be added.]

Disclosure of Interests. The authors have no competing interests to declare.

References

1. Wm A Wulf and Sally A McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH computer architecture news*, 23(1):20–24, 1995.

2. Onur Mutlu. Memory scaling: A systems architecture perspective. In *2013 5th IEEE International Memory Workshop*, pages 21–25. IEEE, 2013.
3. John L Hennessy and David A Patterson. A new golden age for computer architecture. *Communications of the ACM*, 62(2):48–60, 2019.
4. Zhao Zhang, Zhichun Zhu, and Xiaodong Zhang. A permutation-based page interleaving scheme to reduce row-buffer conflicts and exploit data locality. In *Proceedings of the 33rd annual ACM/IEEE international symposium on Microarchitecture*, pages 32–41, 2000.
5. Bruce Jacob, David Wang, and Spencer Ng. *Memory systems: cache, DRAM, disk*. Morgan Kaufmann, 2010.
6. Manu Awasthi, David W Nellans, Kshitij Sudan, Rajeev Balasubramonian, and Al Davis. Handling the problems and opportunities posed by multiple on-chip memory controllers. In *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, pages 319–330, 2010.
7. Muhammad M Rafique, Wei-Ting Lin, Arun Ramanathan, and Rajit Manohar. Dympl: Dynamic multi-feature page management policy using perceptron learning. In *Proceedings of the International Symposium on Memory Systems*, 2022.
8. James M Dodd. Adaptive page management, July 11 2006. US Patent 7,076,617.
9. Nathan Gober, Gino Chacon, Lei Wang, Paul V Gratz, Daniel A Jimenez, Elvira Teran, Seth Pugsley, and Jinchun Kim. The championship simulator: Architectural simulation for education and competition, 2022.
10. Shang Li, Zhiyuan Yang, Dhiraj Reddy, Ankur Srivastava, and Bruce Jacob. DRAMsim3: A cycle-accurate, thermal-capable DRAM simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.
11. Julian Shun and Guy E Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *Proceedings of the 18th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 135–146, 2013.
12. Masab Ahmad, Farrukh Hijaz, Qingchuan Shi, and Omer Khan. CRONO: A benchmark suite for multithreaded graph algorithms executing on futuristic multicores. In *2015 IEEE International Symposium on Workload Characterization*, pages 44–55, 2015.
13. Jure Leskovec and Rok Sosič. SNAP: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology*, 8(1):1–20, 2016.
14. John L Henning. SPEC CPU2006 benchmark descriptions. *ACM SIGARCH Computer Architecture News*, 34(4):1–17, 2006.
15. Timothy Sherwood, Erez Perelman, Greg Hamerly, and Brad Calder. Automatically characterizing large scale program behavior. In *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 45–57, 2002.
16. JEDEC Solid State Technology Association. JEDEC DDR5 SDRAM standard. JESD79-5, 2020.
17. Dimitris Kaseridis, Jeffrey Stuecheli, and Lizy Kurian John. Minimalist open-page: A DRAM page-mode scheduling policy for the many-core era. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 24–35, 2011.
18. Mohsen Ghasempour, Aamer Jaleel, Jim D Garside, and Mikel Luján. HAPPY: Hybrid address-based page policy in DRAMs. In *Proceedings of the Second International Symposium on Memory Systems*, pages 311–321, 2016.

19. Engin Ipek, Onur Mutlu, José F Martínez, and Rich Caruana. Self optimizing memory controllers: A reinforcement learning approach. In *Proceedings of the 35th Annual International Symposium on Computer Architecture*, pages 39–50, 2008.
20. Scott Rixner, William J. Dally, Ujval J. Kapasi, Peter R. Mattson, and John D. Owens. Memory access scheduling. In *27th International Symposium on Computer Architecture (ISCA 2000)*, pages 128–138. IEEE Computer Society, 2000.
21. Onur Mutlu and Thomas Moscibroda. Stall-time fair memory access scheduling for chip multiprocessors. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, pages 146–160. IEEE Computer Society, 2007.
22. Yoongu Kim, Michael Papamichael, Onur Mutlu, and Mor Harchol-Balter. Thread cluster memory scheduling: Exploiting differences in memory access behavior. In *43rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2010)*, pages 65–76. IEEE Computer Society, 2010.
23. Rachata Ausavarungnirun, Kevin Kai-Wei Chang, Lavanya Subramanian, Gabriel H. Loh, and Onur Mutlu. Staged memory scheduling: Achieving high performance and scalability in heterogeneous systems. In *39th International Symposium on Computer Architecture (ISCA 2012)*, pages 416–427. IEEE Computer Society, 2012.
24. Jeffrey Stuecheli, Dimitris Kaseridis, David Daly, Hillery C. Hunter, and Lizy K. John. The virtual write queue: coordinating DRAM and last-level cache policies. In *37th International Symposium on Computer Architecture (ISCA 2010)*, pages 72–82. ACM, 2010.
25. Sai Prashanth Muralidhara, Lavanya Subramanian, Onur Mutlu, Mahmut T. Kandemir, and Thomas Moscibroda. Reducing memory interference in multicore systems via application-aware memory channel partitioning. In *44th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2011)*, pages 374–385. ACM, 2011.
26. Lavanya Subramanian, Vivek Seshadri, Yoongu Kim, Ben Jaiyen, and Onur Mutlu. MISE: providing performance predictability and improving fairness in shared main memory systems. In *19th IEEE International Symposium on High Performance Computer Architecture (HPCA 2013)*, pages 639–650. IEEE Computer Society, 2013.
27. Yoongu Kim, Vivek Seshadri, Donghyuk Lee, Jamie Liu, and Onur Mutlu. A case for exploiting subarray-level parallelism (SALP) in DRAM. In *39th International Symposium on Computer Architecture (ISCA 2012)*, pages 368–379. IEEE Computer Society, 2012.
28. Donghyuk Lee, Yoongu Kim, Vivek Seshadri, Jamie Liu, Lavanya Subramanian, and Onur Mutlu. Tiered-latency DRAM: A low latency and low cost DRAM architecture. In *19th IEEE International Symposium on High Performance Computer Architecture (HPCA 2013)*, pages 615–626. IEEE Computer Society, 2013.
29. Jamie Liu, Ben Jaiyen, Richard Veras, and Onur Mutlu. RAIDR: retention-aware intelligent DRAM refresh. In *39th International Symposium on Computer Architecture (ISCA 2012)*, pages 1–12. IEEE Computer Society, 2012.
30. Aniruddha N. Udipi, Naveen Muralimanohar, Niladrish Chatterjee, Rajeev Balasubramonian, Al Davis, and Norman P. Jouppi. Rethinking DRAM design and organization for energy-constrained multi-cores. In *37th International Symposium on Computer Architecture (ISCA 2010)*, pages 175–186. ACM, 2010.